
DifferLand (Global)

Release 1.1

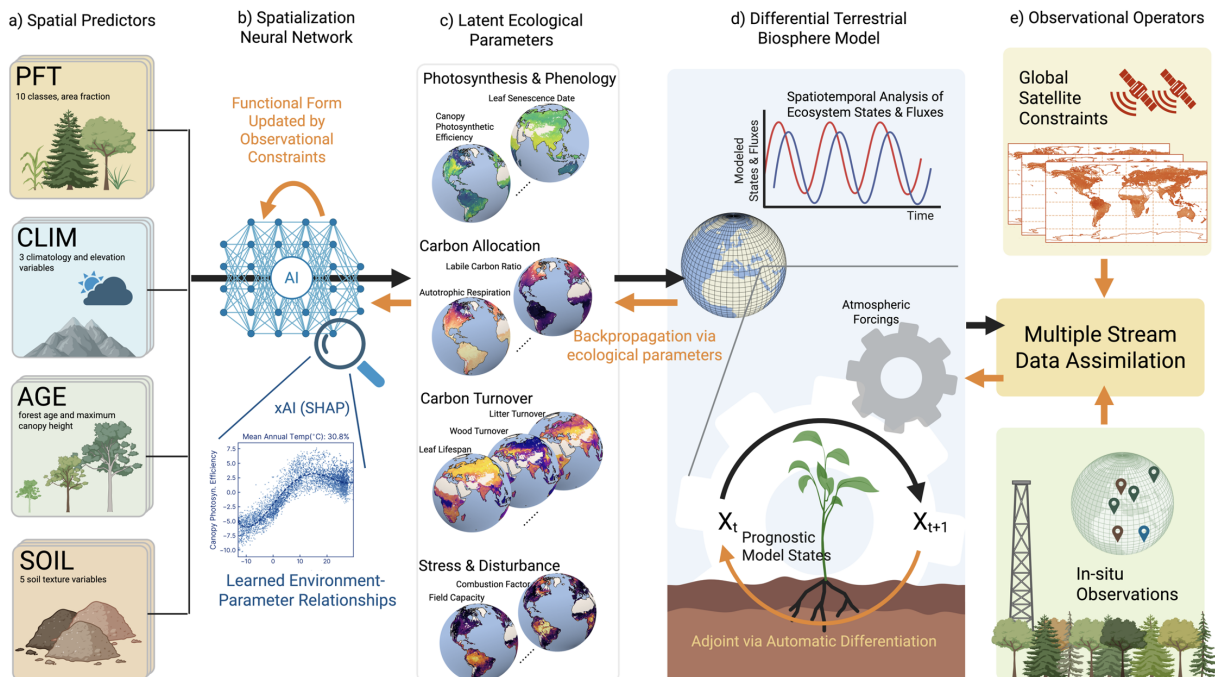
Jianing Fang

Sep 14, 2025

DifferLand is a JAX-based hybrid-machine learning terrestrial biosphere modeling framework designed to learning ecological parameters and functional relationships through multi-modal data assimilation. By leveraging modern automatic differentiation frameworks, DifferLand bridges traditional ecological models with machine learning techniques, enabling hybrid workflows that combine physical understanding with data-driven flexibility. It allows researchers to simulate key ecosystem processes—including photosynthesis, carbon allocation, phenology, turnover, fire disturbance, and water stress—while maintaining transparency and scientific interpretability.

DifferLand offers tools for gradient-based parameter estimation, uncertainty analysis, and exploring how ecosystems respond to environmental variability and change. Its hybrid modeling capabilities open opportunities for integrating ecological knowledge with machine learning to improve predictions and gain new insights into carbon and water dynamics across diverse landscapes.

Users interested in running DifferLand at site-level without parameter spatilization are welcomed to use our [site-level DifferLand model](#).



CONTENTS

1.1 Getting Started

1.1.1 Download DifferLand

DifferLand package is available on [GitHub](https://github.com/JianingFang/DifferLand_Global). You can clone the repository by running:

```
git clone https://github.com/JianingFang/DifferLand_Global.git
```

1.1.2 Repository Structure

- `DifferLand/`: the source code of the DifferLand model.
- `experiments/`: contains the scripts for training the model, saving the results, and computing test statistics and SHAP values.
- `data/`: contains the driver files for running the DifferLand model and analyzing the results. To reproduce the results in the manuscript, you can download the driver files from the [Zenodo repository](#) and place them in this directory.
- `notebooks/`: contains Jupyter notebooks for analyzing results and plotting figures.

1.1.3 Experiment Setup

1. Create a new conda environment and activate it:

```
conda create --name DifferLand python=3.12
conda activate DifferLand
```

2. Install the packages that DifferLand depends on:

```
conda install -c conda-forge jax numpy xarray pandas shap netCDF4 matplotlib
↪cartopy scikit-learn
pip install optax tqdm d
```

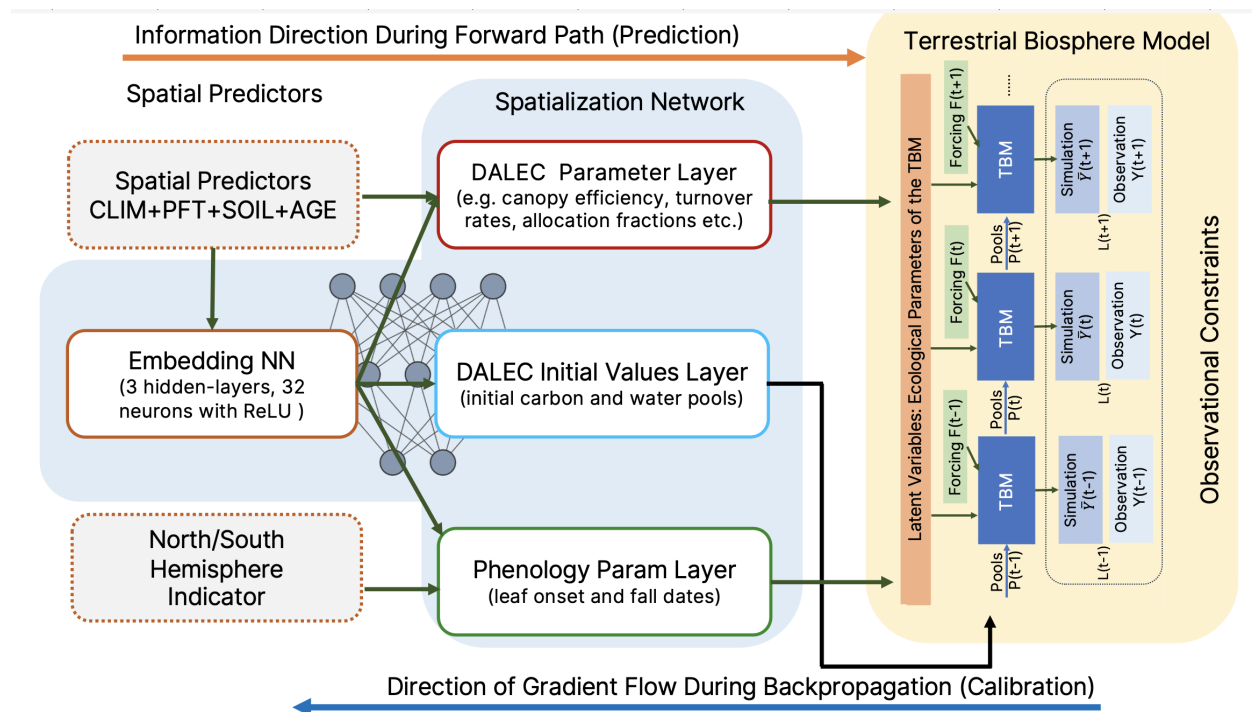
3. Download the necessary driver files from the [Zenodo repository](#) and place them in the `data/` directory.

1.2 DifferLand Overview

1.2.1 Main Components

DifferLand integrates machine learning and process-based ecosystem modeling in a fully differentiable framework. The configuration consists of three main components:

1. **Spatialization Neural Network** ($\mathbf{f}_{nn}$) Maps spatial predictors into ecological parameters, including DALEC process rates, initial pool values, and phenology constants.
2. **Process-Based Terrestrial Biosphere Model** ($\mathbf{f}_M$) A differentiable version of the DALEC model simulating carbon allocation, respiration, turnover, decomposition, phenology, and water fluxes.
3. **Loss Function** ($\mathbf{L}$) Compares simulated outputs against observations (e.g. NEE, LAI, ET, SIF, biomass, SOC) with additional soft constraints to ensure stability.



The detailed design of the spatialization network. The spatialization network (shown in blue) consists of an embedding NN that first uses spatial predictors to learn an embedding of environmental conditions at each 0.25° pixel. The embedding is then passed into three output layers to predict, respectively, the initial conditions ($N=7$, e.g., carbon and water stock in each pool at time zero), the ecological parameters ($N=31$, e.g., allocation fractions to different pools, canopy efficiency, Q10, and turnover rates), and the phenology parameters of the DALEC model ($N=2$, leaf onset & fall days). A North/South hemisphere variable identifies the austral and boreal phenology cycles (with sigmoid activation between $\pm 10^\circ$ to smooth the transition). The predicted initial conditions and model parameters are passed into the differentiable DALEC model to simulate ecosystem dynamics and compute an optimization target based on observational constraints. During the backward pass, the gradient of the neural parameters with respect to the loss function is used to update each layer in the spatialization NN.

1.2.2 Spatialization Network

The neural network has:

- 3 hidden layers with 32 neurons each
- ReLU activation functions
- Xavier initialization for weights (bias = 1)
- **Outputs:**
 - DALEC parameters (31)
 - Initial pools (7)
 - Phenology parameters (2)

A transformation maps parameters from real space into their physical ranges.

Equation (1):

$$\theta_M^k, \theta_i^k = f_{nn}(P^k \mid \theta_{nn})$$

where k denotes pixel index. Parameters θ_{nn} are shared globally.

1.2.3 Process-Based Model

DALEC is implemented in an automatically differentiable form, enabling gradient computation for all parameters and variables.

At each timestep:

Equation (2):

$$\hat{y}_o^{(k,t)}, \hat{y}_u^{(k,t)}, x^{(k,t+1)} = f_M(x^{(k,t)}, m^{(k,t)} \mid \theta_M^k)$$

Substituting Equation (1):

Equation (3):

$$\hat{y}_o^{(k,t)}, \hat{y}_u^{(k,t)} = f_M(m^{(k,1)}, \dots, m^{(k,t)} \mid P^k, \theta_{nn})$$

1.2.4 Loss Function

The loss is a weighted mean squared error across observed variables:

Equation (4):

$$L = \sum_{k,t,v} \alpha_v (\hat{y}_o^{(k,t,v)} - y_o^{(k,t,v)})^2 + L_{constraint}$$

- α_v – weights for each variable
- $L_{constraint}$ – terms penalizing implausible water/carbon dynamics

1.2.5 Implementation

- **Framework:** JAX (automatic differentiation in Python)
- **Design goal:** Parsimonious but mechanistically faithful
- **Outputs:** Differentiable environment-parameter functions informed by spatial predictors, meteorological forcing, and process-based dependencies. These relationships can then be used to predict ecological parameters and produce simulations of global vegetation dynamics.

This design allows DifferLand to balance **interpretability** and **flexibility** in global ecological inference.

1.3 Data Requirements

We provide example driver files at [Zenodo](#).

To run DifferLand, the following input files are required:

- *differland_global_driver_v6.nc*: Contains spatial predictors, atmospheric forcings (except CO₂ concentration), and target variables used to constrain vegetation dynamics.
- *co2_mm_gl_01_23.csv*: Monthly atmospheric CO₂ concentration data.
- *era_valid_v6.nc*: Defines valid land pixels (excluding open water, permanent snow/ice, and sparsely vegetated regions).
- *run_simulation_idx_v6.nc*: Provides random indices for shuffling data and splitting into training, validation, and test sets for ensemble experiments.
- *assimilate_bulk_variable_v6.nc*: Defines patches with sufficient vegetated land area for assimilation of coarse-resolution variables.
- *predictor_mean_v6.csv* and *predictor_std_v6.csv*: Pre-computed mean and standard deviation of predictors for each training split, enabling fast normalization during SHAP computation.

These files are sufficient to reproduce the training and evaluation experiments described in the study.

DifferLand requires three categories of inputs: **spatial predictors** (P), **forcing variables** (m), and **observational constraints** (y). While default datasets are provided below, the framework is flexible and can be readily adapted to incorporate alternative data sources.

1.3.1 Spatial Predictors

Spatial predictors (P) are static variables describing vegetation, climate, soils, and ecosystem structure. They are grouped into four categories:

- **PFT (Plant Functional Types):** - MODIS MCD12C1.v061 fractions aggregated to 0.25° resolution - Consolidated into 11 land cover classes - Pixels dominated by water (>50%) or barren land (>20%) are excluded
- **CLIM (Climate):** - Mean annual temperature (MAT, °C) and precipitation (MAP, mm) from ERA5 (2001–2023) - Elevation (ELE, m) from GTOPO30
- **AGE (Forest Age & Structure):** - Forest age (years) from global ML-based dataset (circa 2010) - Non-forest pixels assigned age = 1 year - Maximum canopy height (CAN.H, m) from ETH Global Sentinel-2 dataset
- **SOIL:** - Bulk density and texture fractions (sand, silt, clay, gravel) - From Harmonized World Soil Database v1.2 - Depth-weighted average over 0–100 cm

All predictors are harmonized to a **0.25° grid** and treated as temporally invariant.

1.3.4 Alternative Data Sources

DifferLand is **flexible** and supports alternative observational streams. Sensitivity tests have included the following substitutions:

- **NBE:** Copernicus CAMS inversion (2001–2023), constrained by ground-based CO₂
- **LAI:** Copernicus LAI (2001–2023), MODIS-independent product
- **Biomass:** IB-AGC (2010–2020), L-VOD–based biomass product
- **VOD:** GLAB-VOD (2003–2020), assimilated at annual level using linear operator
- **Fire emissions:** CMS Carbon Flux for Fire (2010–2016), informed by MOPITT CO

1.3.5 Summary of Default vs. Alternative

OPTIONS	NBE	LAI	Biomass	VOD	Fire Emission
DEFAULT	CMS-Flux (2010–2022)	MODIS LAI v6.1 (2001–2023)	Xu et al. 2020 (2001–2019)	Not used	GFED5 (2001–2020)
ALTERNATIVE	CAMS (2001–2023)	Copernicus LAI (2001–2023)	IB-AGC (2010–2020)	GLAB-VOD (2003–2020)	CMS-Flux Fire (2010–2016)

1.3.6 User Flexibility

Users are encouraged to **experiment with new data streams** as they become available. DifferLand’s modular design allows substitution or addition of observational constraints, enabling the community to evaluate the effects of emerging datasets on model parameterization and predictions.

1.4 Training

1.4.1 Overview

The training driver, `experiments/calibration.py`, calibrates DifferLand end-to-end:

- Spatialization network → maps predictors to ecological parameters
- DALEC TBM → simulates ecosystem dynamics
- Loss function → compares simulations with observations

Training is optimized with **Optax Adam** in JAX.

1.4.2 Required Input Files

Available from the [Zenodo](#) archive:

- differland_global_driver_v6.nc — spatial predictors, forcings, and targets
- co2_mm_gl_01_23.csv — atmospheric CO₂ concentrations
- era_valid_v6.nc — valid land mask
- run_simulation_idx_v6.nc — shuffle/split indices
- assimilate_bulk_variable_v6.nc — assimilation patches

Set the paths in `calibration.py`:

```
DATA_DIR = "/path/to/CARDAMOM_driver_data"
DATA_DIR = os.path.join(DATA_DIR, "global/")
```

1.4.3 Command-Line Options

Option	Default	Description
-r, --run	(required)	Run index (1–100). Controls shuffle/split seed.
-p, --predictors	(required)	Predictor set (e.g. PFT+CLIM+SOIL+AGE).
-n, --neurons	32	Hidden layer width of MLP blocks.
-x, --hidden_layers	3	Number of hidden layers.
-i, --iterations	199	Training iterations.
-l, --learning_rate	5e-5	Adam learning rate.
-t, --number_of_timesteps	23*12	Number of monthly timesteps (years × 12).
-v, --verbose	False	Print progress in addition to logging.

1.4.4 Outputs

- Final parameters: `./output/dalec993_<PREDICTORS>_run_<RUN>.pickle`
- Checkpoints (every 10 iters): `..._checkpoint-<iter>.pickle`
- Logs: `./log/dalec993_<PREDICTORS>_run_<RUN>.log`

1.4.5 How to Run

1. **Prepare data paths** in `calibration.py`.
2. **Run training:**

```
# Minimal example
python calibration.py \
  --run 1 \
  --predictors PFT+CLIM+SOIL+AGE
```

1. **Monitor logs** in `experiments/log/`. Validation loss is printed every 10 iterations.

1.4.6 Adjusting Target Variables

The set of observational constraints used in training can be customized directly in `calibration.py` by editing the target flags under:

```
##### Define the target variables to be assimilated into the framework #####

SIF_PROXY_TARGET = "LCSP"      # LCSP | none
NBE_INVERSION_TARGET = "NBE"    # NBE (CMS-Flux) | CAMS_NBE | none
SATELLITE_LAI_TARGET = "LAI"    # LAI (MODIS) | LAI_COPERNICUS | none
GRACE_LWE_TARGET = "LWE_normalized" # LWE_normalized (GRACE EWT Anomaly) | none
BIOMASS_TARGET = "biomass_yan"  # biomass_yan (Xu et al. 2021) | biomass_ib (Li et al. 2025) | none
FIRE_EMISSION_TARGET = "GFED_FIRE_EMISSION" # GFED_FIRE_EMISSION | fire_emission (CMS-based inversion) | none
STATIC_SOIL_TARGET = "som_const" # som_const (Harmonized World Soil Database) | none
VOD_TARGET = "none"             # none (not assimilated)
GPP_FLUXNET_TARGET = "gpp_fluxnet" # gpp_fluxnet (PFT >25%) | gpp_fluxnet_10percent | gpp_fluxnet_50percent | none
RECO_FLUXNET_TARGET = "reco_fluxnet" # reco_fluxnet (PFT >25%) | reco_fluxnet_10percent | reco_fluxnet_50percent | none
ET_FLUXNET_TARGET = "et_fluxnet" # et_fluxnet (PFT >25%) | et_fluxnet_10percent | et_fluxnet_50percent | none
GLEAM_ET_TARGET = "ET"          # ET (GLEAM) | none
```

Each variable can be **enabled, switched to an alternative dataset, or disabled** by changing its string value.

Examples

- Assimilate only NBE (CMS-Flux) and MODIS LAI:

```
SIF_PROXY_TARGET = "none"
NBE_INVERSION_TARGET = "NBE"
SATELLITE_LAI_TARGET = "LAI"
GRACE_LWE_TARGET = "none"
BIOMASS_TARGET = "none"
FIRE_EMISSION_TARGET = "none"
STATIC_SOIL_TARGET = "none"
GLEAM_ET_TARGET = "none"
VOD_TARGET = "none"
GPP_FLUXNET_TARGET = "none"
RECO_FLUXNET_TARGET = "none"
ET_FLUXNET_TARGET = "none"
```

- Disable all eddy covariance constraints, but use remote sensing and top-down constraints:

```
SIF_PROXY_TARGET = "LCSP"
NBE_INVERSION_TARGET = "NBE"
SATELLITE_LAI_TARGET = "LAI"
GRACE_LWE_TARGET = "LWE_normalized"
BIOMASS_TARGET = "biomass_yan"
FIRE_EMISSION_TARGET = "GFED_FIRE_EMISSION"
STATIC_SOIL_TARGET = "som_const"
```

(continues on next page)

(continued from previous page)

```
GLEAM_ET_TARGET = "ET"
VOD_TARGET = "none"
GPP_FLUXNET_TARGET = "none"
RECO_FLUXNET_TARGET = "none"
ET_FLUXNET_TARGET = "none"
```

This flexibility allows DifferLand to be trained under different combinations of observational streams depending on research needs.

1.4.7 Practical Tips

- Each `--run` index ensures reproducible splits.
- Note that the neural networks however are independently initialized. You may adjust this behavior by setting the seeds in `init_mlp_params`.
- Checkpoints are written every 10 iterations but not auto-resumed.
- Adjust `--number_of_timesteps` to match the driver dataset length.

1.4.8 Example Checkpoints

- Example checkpoints are stored in [Zenodo archive](#) to reproduce the results in the manuscript.

1.5 Evaluate Performance

The script `evaluate_performance.py` is designed to evaluate DifferLand model checkpoints after training. It loads saved neural network and DALEC parameters, computes performance metrics against observational constraints, and exports both latent parameters and simulated ecological variables for downstream analysis.

1.5.1 Main Tasks

1. **Load training checkpoints** - Reads parameter states from the `output/` directory (produced by `calibration.py`). - Supports both final parameter files and intermediate checkpoints.
2. **Compute evaluation metrics** - Compares simulated outputs against the observational datasets assimilated during training (e.g., NEE/NBE, GPP, RECO, LAI, ET, SIF, biomass, SOC, fire emissions). - Reports standard error metrics (e.g., RMSE, correlation, bias) for validation and test sets. The first 2 years are used for warm up and are typically excluded from the evaluation (the length of warm up period (in months) can be adjusted by setting the `--warm_up` option).
3. **Output results for analysis** - Saves latent ecological parameters learned by the spatialization network. - Saves simulated ecological variables (time series and grid-based) from DALEC. - Outputs are written in standard formats for further statistical analysis or visualization (e.g., spatial mapping, temporal trend evaluation).

1.5.2 Usage

Typical usage requires specifying the run index and predictor set to match a previous training run. For example:

```
python evaluate_performance.py \  
--run 1 \  
--predictors PFT+CLIM+SOIL+AGE
```

This will load the corresponding checkpoint (`./output/dalec993_PFT+CLIM+SOIL+AGE_run_1.pickle`), compute metrics, and produce outputs in the `./postanalysis/` directory.

1.5.3 Outputs

- **Metrics:** quantitative performance summaries for validation/test datasets.
- **Latent ecological parameters:** retrieved parameter fields for mapping or sensitivity analysis.
- **Ecological variables:** simulated time series and state/flux variables for subsequent analysis.

These outputs provide the foundation for assessing DifferLand’s predictive skill and for exploring ecological process representations across space and time.

1.6 Ensemble SHAP Analysis

1.6.1 Overview

The script `compute_shap.py` is used to compute **ensemble SHAP values** for DifferLand’s spatialization network. SHAP (SHapley Additive exPlanations) is applied to quantify the marginal contribution of each environmental predictor to the learned ecological parameters. By using the average of an ensemble of model members, the procedure reduces random uncertainties associated with individual neural network initializations.

1.6.2 Methodology

We applied **Kernel SHAP** to extract the environment–parameter relationships learned by the spatialization neural network. Kernel SHAP estimates Shapley values, informed by cooperative game theory, to quantify each feature’s contribution to the model output.

To ensure robustness of SHAP results:

- We defined an **ensemble-averaged model** M_{ens} from **10 out of 20** model members for each configuration that best converged on the training set:

$$M_{ens} = \frac{1}{10} \sum_{i=1}^n M(x, \theta_i)$$

where M is the spatialization network, x are spatial predictors, and θ_i are the neural network weights and biases of member i . Typically we have $n=10$.

- From M_{ens} , **100 grid cells** were sampled to define the background distribution.
- SHAP values were computed on **1000 randomly selected grid cells** within the training dataset.
- **Feature importance** was derived by ranking the mean absolute SHAP values across the 1000 samples.

This procedure is repeated for each selected ecological parameter, providing global trait–environment relationships.

1.6.3 Conditional SHAP by PFT

In addition to global analysis, **conditional SHAP values** can be computed for specific plant functional types (PFTs). This is done by restricting the sampled grid cells to those where a given PFT constitutes at least 80% of the area. This enables evaluation of **ecosystem-specific trait–environment relationships**.

1.6.4 Outputs

The script produces:

- **Global SHAP values:** Feature importances for one ecological parameter at a time.
- **Conditional SHAP values:** Predictor contributions within specific plant functional types.

1.6.5 Usage

A typical workflow involves specifying the ensemble member indices and a single target parameter for SHAP analysis. For example, to compute unconditioned relationships:

```
python compute_shap.py \
  --run 1,2,3,4,5,6,7,8,9,10 \
  --predictors CLIM+SOIL+AGE \
  --target canopy_efficiency
```

Or to condition on a specific plant functional type:

```
python compute_shap.py \
  --run 1,2,3,4,5,6,7,8,9,10 \
  --predictors PFT+CLIM+SOIL+AGE \
  --target canopy_efficiency \
  --pft GRA
```

This will:

1. Load the checkpoints for the specified members.
2. Build the ensemble-averaged model.
3. Compute SHAP values for the specified parameter.
4. Save SHAP arrays in the corresponding directory. These can be later accessed to generate SHAP plots.

1.7 Known Limitations

Despite strong overall performance, the current hybrid DALEC framework has structural limitations that constrain predictive skill in certain processes.

1.7.1 Water Cycle

- **Simplified ET formulation:** Based on fixed water-use efficiency, which does not capture dynamic stomatal regulation under variable meteorological conditions.
- **Missing processes:** - GRACE-derived EWT includes deep groundwater and large-scale hydrology not represented in DALEC's soil-bucket scheme. - Vertical soil moisture transport and groundwater dynamics are absent.
- **Potential improvements:** Incorporating mechanistic stomatal and transpiration formulations (possibly hybrid-ML approaches), multi-layer soil hydrology distinguishing plant-available versus unavailable water, and watershed-scale constraints such as streamflow.

1.7.2 Phenology

- **Fixed leaf dynamics:** DALEC prescribes static onset dates and durations for leaf growth and senescence.
- **Limitations:** - Cannot capture interannual variability in leaf dynamics driven by temperature, soil moisture, vapor pressure deficit, or photoperiod. - Produces realistic mean seasonal cycles of LAI, but does not fully reproduce observed interannual variability in climate-sensitive ecosystems.
- **Potential improvements:** More flexible formulations that couple mechanistic environmental cues with data-driven parameterizations, and explicit representation of leaf turnover and age structure.

1.7.3 Land Use and Land Cover Change

- **Not explicitly represented:** DifferLand currently assumes static land cover and does not account for land use transitions such as deforestation, afforestation, cropland expansion, or urbanization.
- **Implications:** This limits applicability for studies where anthropogenic land use change is a major driver of carbon and water fluxes.
- **Potential improvements:** Coupling with land use change datasets or adding dynamic land cover modules to better capture anthropogenic impacts.

1.7.4 Nutrient Limitation

- **No explicit nutrient cycles:** The model does not include nitrogen or phosphorus cycling.
- **Implications:** Ecosystem processes such as photosynthesis, carbon allocation, and decomposition are simulated without nutrient constraints. This may overestimate productivity in nutrient-limited ecosystems (e.g., tropical forests, boreal soils).
- **Potential improvements:** Incorporating simplified or hybrid-ML nutrient limitation schemes, or coupling with existing nutrient cycle modules, to represent N and P constraints on carbon dynamics.

1.7.5 Future Improvements

These limitations highlight **priority areas for future hybrid land model development**, particularly:

- More mechanistic yet flexible hydrology
- Improved phenology formulations
- Explicit representation of land use and land cover change
- Inclusion of nutrient cycling (N, P)
- Stronger coupling with diverse observational constraints

Such improvements are needed to better capture ecosystem responses under a changing climate.

Also, if you found a bug or would like to suggest an improvement to the code base,
please don't hesitate to contact us at jf3423@columbia.edu or submit a pull request

1.8 Tips for Building Your Model

DifferLand is designed to be modular and extensible. Users can implement new process representations, data sources, and loss terms to tailor the framework to their research needs. Below are recommended practices for building your own model extensions.

1.8.1 Extending DALEC Models

- Implement a new DALEC model version in `DALEC[MODELID].py` by extending the `DALECBASE` class in `DifferLand.model`.
- Use `DALEC993.py` as an example implementation.
- Define the physical boundaries of new parameters and specify any new modeled variables in a corresponding `DALEC_[MODELID]_parinfo.py` file.

1.8.2 Adding New Data

- New datasets can be incorporated by writing them into the `differland_global_driver_vX.nc` file.
- If your new driver data contains **spatial gaps**, you must update the corresponding `era_valid_vX.nc` file to mask out invalid pixels. This prevents propagation of NaN values during training.
- By design, all **meteorological drivers** and **spatial predictors** are expected to be **temporally gap-free**.
- Training targets may contain gaps. The data loading logic in `experiments/calibration.py` can handle missing data through masking.

1.8.3 Customizing Loss Functions

- Additional loss terms can be implemented in `loss_functions.py`.
- To integrate new data into training, update the data loading logic in `experiments/calibration.py` so that the new variables are available during model calibration.

1.8.4 Best Practices

- **Numerical stability:** Be mindful of potential issues such as exploding or vanishing values. Debug with tools like `jax.debug` to locate the source of NaN values during development.
- **Equifinality:** Introducing new processes can increase parameter redundancy. To avoid this, proceed incrementally—validate each modification before adding more complexity.
- **Incremental development:** Start with a minimal working model and expand step by step. Confirm each new addition produces stable and interpretable outputs before moving forward.

By following these guidelines, you can extend DifferLand while maintaining model interpretability, stability, and performance.

1.9 Contribution and Collaboration

We welcome contributions to the **DifferLand** project! Our goal is to build a community around advancing differentiable modeling of the terrestrial ecosystem.

Opportunities for contribution include:

- **Adding new modules** – e.g. alternative spatialization architectures or process-based components.
- **Expanding process descriptions** – improving mechanistic representation of carbon, water, and energy cycles.
- **Including new observational constraints** – assimilating diverse datasets (remote sensing, flux towers, inventories) to strengthen model calibration.

The DifferLand package is publicly available and may be used with proper attribution of the source. If DifferLand represents a substantial component of your research, we warmly encourage you to contact us to discuss your use case and explore opportunities for collaboration.

1.9.1 Collaboration

We are excited about opportunities for collaboration and encourage potential users to reach out. If you are interested in extending DifferLand or collaborating on related projects, please contact:

- **Prof. Pierre Gentine** – pg2328@columbia.edu
- **Jianing Fang** – jf3423@columbia.edu

We look forward to working together to push forward the frontier of **differentiable terrestrial ecosystem modeling**.

1.10 API Reference

1.10.1 DifferLand package

Subpackages

DifferLand.model package

Subpackages

DifferLand.model.auxi package

Submodules

DifferLand.model.auxi.ACM module

`DifferLand.model.auxi.ACM.ACM`(*lat: float | Array, doy: float | Array, t_max: float | Array, t_min: float | Array, lai: float | Array, rad: float | Array, ca: float | Array, ce: float | Array*)

Compute daily Gross Primary Productivity (GPP) using the ACM model.

The ACM (Aggregate Canopy Model) estimates daily GPP from meteorological and vegetation parameters, following a simplified canopy photosynthesis scheme. It accounts for effects of radiation, canopy conductance, CO₂ diffusion, and day length.

Parameters

lat

[Union[float, jnp.ndarray]] Latitude in degrees (decimal), positive for northern hemisphere, negative for southern.

doy

[Union[float, jnp.ndarray]] Day of year (1–365).

t_max

[Union[float, jnp.ndarray]] Daily maximum air temperature (°C).

t_min

[Union[float, jnp.ndarray]] Daily minimum air temperature (°C).

lai

[Union[float, jnp.ndarray]] Leaf Area Index (m² leaf area per m² ground area).

rad

[Union[float, jnp.ndarray]] Daily incoming shortwave radiation (MJ m² day⁻¹).

ca

[Union[float, jnp.ndarray]] CO₂ concentration in the atmosphere (ppm).

ce

[Union[float, jnp.ndarray]] Empirical canopy efficiency parameter, related to leaf nitrogen content, typical range (5–50).

Returns

gpp

[Union[float, jnp.ndarray]] Daily gross primary productivity (GPP) in gC/m2/day

Notes

References: Williams et al. (1997). Predicting gross primary productivity in terrestrial ecosystems. Ecological Applications, 7(3), 882-894.

DifferLand.model.auxi.phenology module

`DifferLand.model.auxi.phenology.lab_release_factor(t, lab_lifespan, clab_release_period, Bday)`

`DifferLand.model.auxi.phenology.leaf_fall_factor(t, leaf_lifespan, leaf_fall_period, Fday)`

`DifferLand.model.auxi.phenology.offset(L, w)`

Module contents

Submodules

DifferLand.model.DALEC993 module

`class DifferLand.model.DALEC993.DALEC993(water_stress_type: str = 'default')`

Bases: [DALECBase](#)

DALEC993 model implementation.

This class implements the DALEC993 variant of the DALEC model, defining specific parameter bounds, phenology parameters, and model stepping logic.

Attributes

water_stress_type

[str, optional] Type of water stress formulation to use. Supported options include:

- “nothing”: No water stress is applied.
- “default”: Use the default water stress formulation.
- “nn_paw”: Neural-network-based water stress using plant available water.
- “nn_whole”: Neural network estimates water stress using full inputs.
- “nn_whole_no_lai”: Similar to `nn_whole` but excludes LAI from inputs.
- “gpp_acm_et_nn”: Neural network determines GPP and ET water stress jointly.

Default is “default”.

parmin

[DALEC_993_parinfo.DALEC993ParamBounds] Lower bounds for model parameters.

parmax

[DALEC_993_parinfo.DALEC993ParamBounds] Upper bounds for model parameters.

pfn

[DALEC_993_parinfo.DALEC993Outputs] Data structure defining model outputs.

pheno_parma

[jnp.ndarray] Maximum allowed values for phenology parameters.

pheno_parmin

[jnp.ndarray] Minimum allowed values for phenology parameters.

parmax: *DALEC993ParamBounds*

parmin: *DALEC993ParamBounds*

pfn: *DALEC993Outputs*

pheno_parma: *Array*

pheno_parmin: *Array*

step(*pools: Array, met: Array, dalec_parameters: Array, pheno_parameters: Array, gpp_params: Array | None = None*) → Tuple[Array, Array]

Perform a single forward time step of the DALEC993 model.

This method computes ecosystem carbon fluxes and updates the carbon pool states for one timestep, based on meteorological inputs and model parameters.

Parameters

pools

[jnp.ndarray] Current ecosystem carbon pools. Typical pools may include foliage, wood, litter, soil organic matter, etc. Shape: (n_pools,).

met

[jnp.ndarray] Meteorological forcing data for the current timestep. Examples include temperature, precipitation, radiation, etc. Shape: (n_met_vars,).

dalec_parameters

[jnp.ndarray] Array of DALEC model parameters controlling carbon flux rates and physiological processes. Shape: (n_dalec_parameters,).

pheno_parameters

[jnp.ndarray] Phenology parameters that modulate seasonal dynamics and flux partitioning. Shape: (n_pheno_parameters,).

gpp_params

[Optional[jnp.ndarray], optional] Additional parameters for Gross Primary Production (GPP) modeling, if applicable. Shape depends on the chosen water stress or GPP scheme. Default is None.

Returns

new_pools

[jnp.ndarray] Updated ecosystem carbon and water pools after applying fluxes for the current timestep.
Shape: (self.n_pool,).

new_output

[jnp.ndarray] Updated pools and fluxes or diagnostic outputs computed during the timestep, such as GPP, respiration, litterfall, etc. Shape: (self.n_output)

Notes

- This method is JIT-compiled for performance using *jax.jit*.
- The exact flux calculations depend on the chosen water stress formulation

and internal parameterization.

unnormalize_pools(*normalized_pools: Array*) → Array

Map pool parameters from unconstrained real space to their physical range.

This method transforms phenology parameters, which may exist in an unconstrained real-valued space, back into their bounded physical range defined by model-specific minimum and maximum phenology parameter values.

Parameters

normalized_pools

[jnp.ndarray] Array of pool parameters in real-valued space (not yet constrained to the physical parameter bounds).

Returns

jnp.ndarray

Array of pool parameters transformed into the physical range defined between *pheno_parmin* and *pheno_parmax*.

unnormalize_water(*normalized_parameters: Array*) → Array

Map water-related parameters from unconstrained real space to their physical range.

This method transforms water-related model parameters, which may exist in an unconstrained real-valued space (e.g., outputs of an optimizer or neural network), back into their bounded physical range as defined by parameter minimum and maximum values.

For water-related parameters, the physical range is defined by:

- Minimum value: 1.0
- Maximum value: 10,000.0

Parameters

normalized_parameters

[jnp.ndarray] Array of water-related parameters in real-valued space (not yet constrained to the physical parameter bounds).

Returns

jnp.ndarray

Array of water-related parameters transformed into the physical range.

water_stress_type: `str = 'default'`

DifferLand.model.DALECBase module

class DifferLand.model.DALECBase.DALECBase

Bases: object

Base class for the DALEC model framework.

This class defines the interface for DALEC models, including the required methods to be implemented in any derived class.

Methods

forward(*args, **kwargs)

Run the model forward in time.

unnormalize_parameters(*args, **kwargs)

Convert normalized parameters back to their original scale.

step(*args, **kwargs)

Perform a single time step of the model.

forward(*args, **kwargs)

Run the model forward in time.

Parameters

***args :**

Positional arguments for the forward model run.

****kwargs :**

Keyword arguments for the forward model run.

Raises

NotImplementedError

If the method is not implemented in a derived class.

id: `Array`

param_parmax: `Array`

param_parmin: `Array`

pool_parmax: `Array`

pool_parmin: `Array`

step(**args, **kwargs*)

Perform a single time step of the model.

Parameters

***args :**

Positional arguments for the model step.

****kwargs :**

Keyword arguments for the model step.

Raises

NotImplementedError

If the method is not implemented in a derived class.

unnormalize(*normalized_parameters*)

unnormalize_pheno(*normalized_pools*)

DifferLand.model.DALEC_993_parinfo module

```

class DifferLand.model.DALEC_993_parinfo.DALEC993Outputs(lai, gpp, ET, temperate, respiration_auto,
    leaf_production, labile_production,
    root_production, wood_production, lff,
    lrf, labile_release, leaf_litter,
    wood_litter, root_litter,
    respiration_hetero_litter,
    respiration_hetero_som, litter_to_som,
    runoff, labile_fire_combust,
    foliar_fire_combust, root_fire_combust,
    wood_fire_combust, litter_fire_combust,
    som_fire_combust, labile_fire_transfer,
    foliar_fire_transfer, root_fire_transfer,
    wood_fire_transfer, litter_fire_transfer,
    total_fire_combust, nee, nbe,
    next_labile_pool, next_foliar_pool,
    next_root_pool, next_wood_pool,
    next_litter_pool, next_som_pool,
    next_water_pool, beta, SIF, violation,
    vod)

```

Bases: tuple

ET

Alias for field number 2

SIF

Alias for field number 41

beta

Alias for field number 40

foliar_fire_combust

Alias for field number 20

foliar_fire_transfer

Alias for field number 26

gpp

Alias for field number 1

labile_fire_combust

Alias for field number 19

labile_fire_transfer

Alias for field number 25

labile_production

Alias for field number 6

labile_release

Alias for field number 11

lai

Alias for field number 0

leaf_litter

Alias for field number 12

leaf_production
Alias for field number 5

lff
Alias for field number 9

litter_fire_combust
Alias for field number 23

litter_fire_transfer
Alias for field number 29

litter_to_som
Alias for field number 17

lrf
Alias for field number 10

nbe
Alias for field number 32

nee
Alias for field number 31

next_foliar_pool
Alias for field number 34

next_labile_pool
Alias for field number 33

next_litter_pool
Alias for field number 37

next_root_pool
Alias for field number 35

next_som_pool
Alias for field number 38

next_water_pool
Alias for field number 39

next_wood_pool
Alias for field number 36

respiration_auto
Alias for field number 4

respiration_hetero_litter
Alias for field number 15

respiration_hetero_som
Alias for field number 16

root_fire_combust
Alias for field number 21

root_fire_transfer
Alias for field number 27

root_litter

Alias for field number 14

root_production

Alias for field number 7

runoff

Alias for field number 18

som_fire_combust

Alias for field number 24

temperate

Alias for field number 3

total_fire_combust

Alias for field number 30

violation

Alias for field number 42

vod

Alias for field number 43

wood_fire_combust

Alias for field number 22

wood_fire_transfer

Alias for field number 28

wood_litter

Alias for field number 13

wood_production

Alias for field number 8

```
class DifferLand.model.DALEC_993_parinfo.DALEC993ParamBounds(decomposition_rate, f_gpp, f_fol,
f_root, leaf_lifespan, tor_wood,
tor_root, tor_litter, tor_som, Q10,
canopy_efficiency, Bday, flab,
clab_release_period, Fday,
leaf_fall_period, LCMA, Clab, Cfol,
Croot, Cwood, Clitter, Csom, IWUE,
runoff_focal_point, field_capacity,
initial_water, foliar_cf, ligneous_cf,
dom_cf, resilience, lab_lifespan,
moisture_factor, uWUE, boese_r,
wilting_point_frac, sif_alpha,
sif_beta_plus_three, p_fol, p_wood)
```

Bases: tuple

Bday

Alias for field number 11

Cfol

Alias for field number 18

Clab

Alias for field number 17

Clitter

Alias for field number 21

Croot

Alias for field number 19

Csom

Alias for field number 22

Cwood

Alias for field number 20

Fday

Alias for field number 14

IWUE

Alias for field number 23

LCMA

Alias for field number 16

Q10

Alias for field number 9

boese_r

Alias for field number 34

canopy_efficiency

Alias for field number 10

clab_release_period

Alias for field number 13

decomposition_rate

Alias for field number 0

dom_cf

Alias for field number 29

f_fol

Alias for field number 2

f_gpp

Alias for field number 1

f_root

Alias for field number 3

field_capacity

Alias for field number 25

flab

Alias for field number 12

foliar_cf

Alias for field number 27

initial_water

Alias for field number 26

lab_lifespan

Alias for field number 31

leaf_fall_period

Alias for field number 15

leaf_lifespan

Alias for field number 4

ligneous_cf

Alias for field number 28

moisture_factor

Alias for field number 32

p_fol

Alias for field number 38

p_wood

Alias for field number 39

resilience

Alias for field number 30

runoff_focal_point

Alias for field number 24

sif_alpha

Alias for field number 36

sif_beta_plus_three

Alias for field number 37

tor_litter

Alias for field number 7

tor_root

Alias for field number 6

tor_som

Alias for field number 8

tor_wood

Alias for field number 5

uWUE

Alias for field number 33

wilting_point_frac

Alias for field number 35

Module contents

DifferLand.optimization package

Submodules

DifferLand.optimization.forward module

DifferLand.optimization.forward.**embed_prediction_forward**(*params: List[Dict[str, Array]]*,
predictors: Array) → Array

Forward pass through a multi-layer perceptron to produce embeddings.

Each hidden layer applies a linear transformation followed by a leaky ReLU activation. The final layer also applies a linear transformation followed by a leaky ReLU activation.

Parameters

params

[list of dict] A list of layer parameter dictionaries. Each dictionary has:

- ‘weights’ : jax.numpy.ndarray of shape (n_in, n_out)
- ‘biases’ : jax.numpy.ndarray of shape (n_out,)

predictors

[jax.numpy.ndarray] Input feature matrix of shape (batch_size, n_features).

Returns

jax.numpy.ndarray

Embedding output of shape (batch_size, n_outputs).

DifferLand.optimization.forward.**parameter_prediction_forward**(*params: List[Dict[str, Array]]*,
predictors: Array) → Array

Forward pass through a multi-layer perceptron to produce parameter predictions.

Each hidden layer applies a linear transformation followed by a leaky ReLU activation. The final layer applies only a linear transformation (no activation).

Parameters

params

[list of dict] A list of layer parameter dictionaries. Each dictionary has:

- ‘weights’ : jax.numpy.ndarray of shape (n_in, n_out)
- ‘biases’ : jax.numpy.ndarray of shape (n_out,)

predictors

[jax.numpy.ndarray] Input feature matrix of shape (batch_size, n_features).

Returns

`jax.numpy.ndarray`

Output predictions of shape (batch_size, n_outputs).

DifferLand.optimization.loss_functions module

`DifferLand.optimization.loss_functions.compute_edc_loss`(*result*: Array, *initial_pools_sample*: Array, *pfn*: Any, *epsilon*: float = 0.001) → Array

Compute the Ecosystem Disequilibrium Cost (EDC) loss.

This function calculates a squared log-ratio loss measuring how far simulated carbon pool sizes and fluxes deviate from initial conditions, accounting for internal ecosystem fluxes and average seasonal pools.

It uses the ratio of mean annual influx to outflux for each carbon pool, adjusted by initial pool sizes and seasonal means. The resulting loss penalizes imbalances in the modeled carbon cycle and helps maintain mass balance constraints during model optimization.

Parameters

result

[`jax.numpy.ndarray`] Array of shape (n_samples, n_timesteps, n_outputs) containing model outputs such as carbon fluxes and pools for each sample and timestep.

initial_pools_sample

[`jax.numpy.ndarray`] Array of shape (n_samples, n_pools) with initial carbon pool sizes for each sample. The columns correspond to: labile, leaf, root, wood, litter, and SOM pools.

pfn

[Any] Object or module providing integer indices into the third axis of *result* for various fluxes and pools. Must contain attributes such as:

- `labile_production`
- `labile_release`
- `labile_fire_combust`
- `labile_fire_transfer`
- `leaf_production`
- `leaf_litter`
- `foliar_fire_transfer`
- `foliar_fire_combust`
- `wood_production`
- `wood_litter`
- `wood_fire_combust`
- `wood_fire_transfer`
- `root_production`
- `root_litter`
- `root_fire_combust`

- root_fire_transfer
- respiration_hetero_litter
- litter_to_som
- litter_fire_combust
- litter_fire_transfer
- respiration_hetero_som
- som_fire_combust
- next_labile_pool
- next_foliar_pool
- next_root_pool
- next_wood_pool
- next_litter_pool
- next_som_pool

epsilon

[float, optional] Small value to avoid division by zero or taking logarithm of zero (default is 1e-3).

Returns**jax.numpy.ndarray**

Array of shape (n_samples,) containing the EDC loss for each sample.

Notes

- The loss penalizes deviations of modeled pools from initial values using squared log2 differences.
- The January means are computed using timesteps 11, 23, 35, etc., assuming monthly model outputs and January being timestep index 11 in a zero-based indexing system.

`DifferLand.optimization.loss_functions.loss_fn_with_edc`(*params: Array, predictors: Array, met: Array, labels: Array, batch_forward: Callable[[Array, Array, Array], tuple[Array, Array]], pfn: Any, warm_up: int = 0, batch_size: int = 320, epsilon: float = 0.001*) → float | Array

Compute the total loss for the model, including Ecological and Dynamical Constraints (EDC).

This loss function evaluates how well the model reproduces various observed variables (fluxes, stocks, and other ecological quantities) and applies penalties for violations of ecological and dynamical constraints.

The total loss includes the following components: - SIF loss (Solar-Induced Fluorescence) - LAI loss (Leaf Area Index) - Biomass loss (annual) - NBE loss (Net Biome Exchange) - NBE interannual variability loss - Fire loss - SOM loss (Soil Organic Matter) - Water pool loss - VOD loss (Vegetation Optical Depth, annual) - FluxNet GPP, RECO, and ET losses - GLEAM ET loss - Violation loss (sum of violations predicted by the model) - EDC loss

Parameters

params

[Array] Model parameters to be evaluated. These are typically the normalized parameters being optimized.

predictors

[Array] Predictor variables used by the model, e.g. climate drivers, site variables.

met

[Array] Meteorological forcing data for the model.

labels

[Array] Observational data to compare model outputs against. This includes: - fluxes - stocks - masks and relative weights indicating valid observations

batch_forward

[Callable[[Array, Array, Array], tuple[Array, Array]]]

Function that performs a forward simulation of the model:

`initial_pools, result = batch_forward(params, predictors, met)`

- *initial_pools* : Array of model state pools at start.
- *result* : Array of model outputs with shape (batch_size, time, variables).

pfn

[Any] Object holding index mappings into *result* for different output variables. E.g. pfn.SIF, pfn.lai, etc.

warm_up

[int, optional] Number of initial timesteps to skip from loss computation (default is 0). Used to avoid initial model spin-up effects.

batch_size

[int, optional] Number of samples in the batch. Default is 320.

epsilon

[float, optional] Small constant to avoid division by zero in normalization operations.

Returns

total_loss

[float or Array] Total computed loss value, combining all error components and EDC penalties.

Notes

- The function aggregates multiple error terms using different weights.
- EDC (Ecological and Dynamical Constraints) helps enforce ecological realism and system stability in the model by penalizing physically or biologically implausible states or parameter combinations.

`DifferLand.optimization.loss_functions.patch_of_error`(*predicted*: Array, *true*: Array, *mask*: Array, *weight*: int | float = 1) → Array

Compute the weighted patch-level squared error between predicted and true values.

This measures how the spatial mean of predictions deviates from the mean of true values in each feature (or spatial column), weighted by the average mask coverage.

Parameters

predicted

[jax.numpy.ndarray] Predicted values, of shape (n_samples, n_features).

true

[jax.numpy.ndarray] True target values, of the same shape as *predicted*.

mask

[jax.numpy.ndarray] Mask array of the same shape as *predicted* and *true*. Elements with mask value 0 are excluded from the mean calculations.

weight

[int or float, optional] Scalar multiplier applied to the final patch error (default is 1).

Returns

jax.numpy.ndarray

Scalar tensor representing the weighted patch-level squared error.

DifferLand.optimization.loss_functions.**sum_of_error**(*predicted*: Array, *true*: Array, *mask*: Array, *weight*: int | float = 1) → Array

Compute the weighted sum of squared errors between predicted and true values, considering only entries where the mask is nonzero.

Parameters

predicted

[jax.numpy.ndarray] Predicted values.

true

[jax.numpy.ndarray] True target values.

mask

[jax.numpy.ndarray] Mask array of the same shape as *predicted* and *true*. Elements with mask value 0 are excluded from the error calculation.

weight

[int or float, optional] Scalar multiplier applied to the final summed error (default is 1).

Returns

jax.numpy.ndarray

Scalar tensor representing the weighted sum of squared errors.

Module contents

Submodules

DifferLand.DifferLand module

`class DifferLand.DifferLand.DifferLand`

Bases: `object`

Module contents

1.11 Citations

Fang, J., Bowman, K., Zhao, W., Lian, X., & Gentine, P. (2024). Differentiable Land Model Reveals Global Environmental Controls on Ecological Parameters. arXiv preprint arXiv:2411.09654.

Fang, J., & Gentine, P. (2024). Exploring Optimal Complexity for Water Stress Representation in Terrestrial Carbon Models: A Hybrid-Machine Learning Model Approach. Journal of Advances in Modeling Earth Systems, 16(12), e2024MS004308.

1.12 Contacts

Jianing Fang (jf3423@columbia.edu), Pierre Gentine (pg2328@columbia.edu)